week5_notebook_student

October 28, 2026

1 HS4002 Week 5

1.1 Basics of OLS Regression

Today we'll learn linear regression through one research question: **to what extent does job prestige predict income?**

We'll build three models: - **Model 1**: income ~ prestige - **Model 2**: income ~ prestige + gender - **Model 3**: income ~ prestige + gender + age

In Python, `statsmodels` provides OLS regression with a **formula API** that mirrors R's `lm()` syntax almost exactly.

[1]:
```
# !pip install pandas numpy plotnine statsmodels stargazer

import pandas as pd
import numpy as np
from plotnine import *
import warnings
warnings.filterwarnings('ignore')
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm
```

1.2 Data Setup

[2]:
```
raw_df = pd.read_csv("gss2022_mini.csv")

chosen_variables = [
        'id', 'age', 'yrlvmus', 'yrartxbt', 'yrmovie', 'yrcreat',
        'income16', 'prestg10', 'degree', 'race', 'sex', 'wrkstat'
]
df = raw_df[chosen_variables].dropna().copy()

# Recode income16 to continuous midpoint values
income_map = {
        1: 500,    2: 2000,   3: 3500,   4: 4500,   5: 5500,   6: 6500,
        7: 7500,   8: 9000,   9: 11250,  10: 13750, 11: 16250, 12: 18750,
        13: 21250, 14: 23750, 15: 27500, 16: 32500, 17: 37500, 18: 45000,
        19: 55000, 20: 67500, 21: 82500, 22: 100000, 23: 120000, 24: 140000,
```

```
        25: 160000, 26: 250000
}
df['income_cont'] = df['income16'].map(income_map)

# Binary cultural participation indicators
df['binary_lvmus']  = (df['yrlvmus']  == 1).astype(int)
df['binary_artxbt'] = (df['yrartxbt'] == 1).astype(int)
df['binary_movie']  = (df['yrmovie']  == 1).astype(int)
df['binary_creat']  = (df['yrcreat']  == 1).astype(int)
df['omni'] = df[['binary_lvmus','binary_artxbt','binary_movie','binary_creat']].
 ↪sum(axis=1)

# Derived variables
df['race_bin']   = df['race'].astype(str)
df['ba_binary']  = (df['degree'] >= 3).astype(int)
df['sex_woman']  = np.where(df['sex'] == 2, 'Woman', 'Not Woman')
df['working']    = np.where(df['wrkstat'] <= 3, 'Working', 'Not Working')
```

2 Model 1

Regress income (`income_cont`) on occupational prestige (`prestg10`).

Hint: `smf.ols('y ~ x', data=df).fit()` is the Python equivalent of `lm(y ~ x, data=df)` in R.

[3]:
```
lm1 = smf.ols('income_cont ~ prestg10', data=df).fit()
# regress income (y) on occupational prestige (x)
```

2.1 Print a summary of `lm1`

Equivalent to R's `summary(lm1)`.

[4]:
```
print(lm1.summary())
# lm1 is the model from above, this shows the model to you
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:            income_cont   R-squared:                       0.175
Model:                            OLS   Adj. R-squared:                  0.174
Method:                 Least Squares   F-statistic:                     149.7
Date:                Tue, 27 Oct 2026   Prob (F-statistic):           2.42e-31
Time:                        22:06:13   Log-Likelihood:                -8842.4
No. Observations:                 707   AIC:                         1.769e+04
Df Residuals:                     705   BIC:                         1.770e+04
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
```

```
------------------------------------------------------------------------------
Intercept  -2.001e+04   9120.202     -2.194      0.029   -3.79e+04   -2103.508
prestg10    2236.5640    182.823     12.233      0.000    1877.621    2595.507
==============================================================================
Omnibus:                      106.536   Durbin-Watson:                   1.693
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              153.957
Skew:                           1.071   Prob(JB):                     3.70e-34
Kurtosis:                       3.797   Cond. No.                         185.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

2.1.1 Provide a substantive interpretation of the results from `lm1`.

When prestige = 0, predicted income = -2.001e. What the 2236.54 means: for every 1 point increase in prestige, your income is expected to increase by 2236.65, holding all else constant. Since p-value = 0, there is strong evidence to suggest that prestige impacts income. there is a 95% confidence interval that every 1 point increase in prestige causes an increase in income between $1977 and $2595.

since R-squared = 0.175, the model explains 17.5% of variation in income; Since p-value for f-statistic is small, there is strong evidence to suggest that prestige impacts income.

omnibus and jarque-bera are tests on if the residuals form a bell curve. since p-value for omnibus = 0 and JB is far from 0, data does not follow a normal distribution. since skew > 0, the curve is positively skewed, with a long tail to the right. some people earn way more than predicted. since kurtosis > 3, the curve has slightly more extreme values than normal.

3 Model 2

Add gender (`sex_woman`) to the model. `statsmodels` automatically creates dummy variables for string/categorical columns.

[6]:
```
lm2 = smf.ols('income_cont ~ prestg10 + sex_woman', data=df).fit()
print(lm2.summary())

# if you are a women, you are expected to have a $10000 smaller income, holding␣
 ↪all else constant
# p-value of 0.034 < 0.05, which is very small. there is strong evidence to␣
 ↪suggest that sex affects income
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:            income_cont   R-squared:                       0.180
Model:                            OLS   Adj. R-squared:                  0.178
Method:                 Least Squares   F-statistic:                     77.45
Date:                Tue, 27 Oct 2026   Prob (F-statistic):           3.96e-31
```

```
Time:                        23:18:27   Log-Likelihood:                -8840.2
No. Observations:                 707   AIC:                         1.769e+04
Df Residuals:                     704   BIC:                         1.770e+04
Df Model:                           2
Covariance Type:            nonrobust
================================================================================
======
                          coef    std err          t      P>|t|      [0.025
0.975]
--------------------------------------------------------------------------------
------
Intercept          -1.381e+04   9556.103     -1.445      0.149   -3.26e+04
4956.353
sex_woman[T.Woman] -1.044e+04   4922.158     -2.121      0.034   -2.01e+04
-778.365
prestg10            2221.1749    182.515     12.170      0.000    1862.836
2579.514
==============================================================================
Omnibus:                      107.120   Durbin-Watson:                   1.690
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              155.350
Skew:                           1.071   Prob(JB):                     1.85e-34
Kurtosis:                       3.827   Cond. No.                         197.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

3.1 Compare Model 1 and Model 2

3.1.1 Comparing R^2 and Adjusted R^2 values

[ ]:
```
comparison = pd.DataFrame({
        'Model':          ['Model 1', 'Model 2'],
        'R-Squared':      [lm1.rsquared,     lm2.rsquared],
        'Adj. R-Squared':[lm1.rsquared_adj, lm2.rsquared_adj]
})
comparison

# model 2 explauns 18% of variation in income, which is more than model 1
# model 2 has a slightly higher adjusted R-squared than model 1. adding gender␣
 ↪helps a little
# thus model 2 is slightly better than model 1 in explaining variation in income
```

[ ]:

	Model	R-Squared	Adj. R-Squared
0	Model 1	0.175109	0.173939
1	Model 2	0.180349	0.178020

3.1.2 Comparing AIC values

Lower AIC is better.

[ ]:
```
print(f'AIC Model 1: {lm1.aic:.2f}')
print(f'AIC Model 2: {lm2.aic:.2f}')

# compare the AIC of 2 models, and see which one is lower
# model 2 has a slightly lower AIC than model 1. model 2 fits the data slightly␣
 ↪better, but the difference from adding gender is small
```

```
AIC Model 1: 17688.88
AIC Model 2: 17686.38
```

3.1.3 Conducting an F-test

Equivalent to R's `anova(lm1, lm2)`.

This works only for **nested** models — Model 1's predictors must be a subset of Model 2's, fitted on the same rows. It cannot compare `prestige + gender` against `prestige + age`, since neither one contains the other. Remember, the null hypothesis tests if the additional variables in the unrestricted model **all** have coefficients of zero.

[ ]:
```
# anova_lm compares nested models with an F-test
# use f-test tests to determine if adding another variable (i.e. sex)␣
 ↪statistically significant
# checks if adding a new variable helps shrink the mistakes (i.e. residuals:␣
 ↪gap between predicted income and actual income)
print(anova_lm(lm1, lm2))

# ssr is the total mistake score. model 2 has less istakes than model 1
# df_diff is 1 for model 2 as it has one added variable
# ss_diff if how much the variables shrank
# as p = 0.034 < 0.05, adding sex significantly reduces the residual sum of␣
 ↪squares, F(1, 704) = 4.50.
```

```
   df_resid           ssr  df_diff       ss_diff         F    Pr(>F)
0     705.0  3.022575e+12      0.0           NaN       NaN       NaN
1     704.0  3.003374e+12      1.0  1.920049e+10  4.500653  0.034231
```

3.1.4 Which model do you prefer? Justify using the statistics above.

[ ]:
```
# Model 2 is preferred, as it has a lower AIC and scoore, a higher R-squared␣
 ↪score, and fits better than modele 1 according to the results of the F-test.
```

4 Model 3

Add age (`age`) to Model 2.

[9]:
```
lm3 = smf.ols('income_cont ~ prestg10 + sex_woman + age', data=df).fit()
print(lm3.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:            income_cont   R-squared:                       0.181
Model:                            OLS   Adj. R-squared:                  0.178
Method:                 Least Squares   F-statistic:                     51.91
Date:                Wed, 28 Oct 2026   Prob (F-statistic):           2.57e-30
Time:                        02:32:46   Log-Likelihood:                -8839.8
No. Observations:                 707   AIC:                         1.769e+04
Df Residuals:                     703   BIC:                         1.771e+04
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      -7449.5499   1.17e+04     -0.634      0.526   -3.05e+04    1.56e+04
sex_woman[T.Woman] -1.067e+04   4928.630     -2.165      0.031   -2.03e+04   -992.318
prestg10        2234.3835    183.082     12.204      0.000    1874.930    2593.837
age             -132.5390    142.270     -0.932      0.352    -411.863     146.785
==============================================================================
Omnibus:                      106.908   Durbin-Watson:                   1.683
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              154.919
Skew:                           1.070   Prob(JB):                     2.29e-34
Kurtosis:                       3.824   Cond. No.                         350.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

4.1 Compare Model 2 and Model 3

[ ]:
```
comparison2 = pd.DataFrame({
        'Model':           ['Model 2', 'Model 3'],
        'R-Squared':       [lm2.rsquared,     lm3.rsquared],
        'Adj. R-Squared': [lm2.rsquared_adj, lm3.rsquared_adj],
        'AIC':             [lm2.aic,          lm3.aic]
})
```

```
comparison2
# model 3 has a higher R-squared value than model 2
# but model 3 has a lower adjusted R-squared value. adding age does not help in␣
 ↪explaining variation in income
# model 2 has a lower AIC score, so it fits the data better
```

[ ]:

	Model	R-Squared	Adj. R-Squared	AIC
0	Model 2	0.180349	0.178020	17686.375978
1	Model 3	0.181360	0.177866	17687.503692

[ ]:
```
print(anova_lm(lm2, lm3))
# as p-value > 0.35, adding age is not stastically significant
# model 2 is better
```

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	704.0	3.003374e+12	0.0	NaN	NaN	NaN
1	703.0	2.999671e+12	1.0	3.703232e+09	0.867886	0.351861

5 Making a professional-looking regression table

The Python `stargazer` package produces LaTeX/HTML tables equivalent to R's `stargazer` or `texreg`.

Run the cell below, then follow the Overleaf steps to turn that output into a typeset PDF table.

5.1 How to use Overleaf

Overleaf is a LaTeX editor that runs in your browser. There is nothing to install, it compiles your document to PDF for you, and teammates can edit the same file at once. You'll be writing the research paper in it, so get comfortable now.

5.1.1 1. Set up your project

1. Go to overleaf.com and register for a free account.
2. Open the course template: https://www.overleaf.com/read/hqjkhwkfnfbr#d8258f
3. That link is **read-only**, so you can't type in it. Click **Menu → Copy Project** to get your own editable copy. (If that fails, click **New Project → Blank Project** and paste the template's text in yourself.)

5.1.2 2. Paste the table in

Copy the whole block the cell above printed — everything from `\begin{table}` down to `\end{table}` — and paste it between `\begin{document}` and `\end{document}`. Then click **Recompile**.

You'll have made your first LaTeX table!

[12]:
```
from stargazer.stargazer import Stargazer

sg = Stargazer([lm1, lm2, lm3])
```

```python
sg.title('Regression Results')

# LaTeX reads an underscore as a subscript, so the raw name 'income_cont'
# would stop the document from compiling in Overleaf.
sg.dependent_variable_name('Income (USD)')

sg.custom_columns(['Model 1', 'Model 2', 'Model 3'], [1, 1, 1])
sg.covariate_order(['prestg10', 'sex_woman[T.Woman]', 'age', 'Intercept'])
sg.rename_covariates({
        'prestg10':           'Occupational prestige',
        'sex_woman[T.Woman]': 'Gender (woman = 1)',
        'age':                'Age',
        'Intercept':          'Intercept'
})

# Render as LaTeX - paste into Overleaf
print(sg.render_latex())
```

```
\begin{table}[!htbp] \centering
  \caption{Regression Results}
\begin{tabular}{@{\extracolsep{5pt}}lccc}
\\[-1.8ex]\hline
\hline \\[-1.8ex]
& \multicolumn{3}{c}{\textit{Dependent variable: Income (USD)}} \
\cr \cline{2-4}
\\[-1.8ex] & \multicolumn{1}{c}{Model 1} & \multicolumn{1}{c}{Model 2} &
\multicolumn{1}{c}{Model 3}  \\
\\[-1.8ex] & (1) & (2) & (3) \\
\hline \\[-1.8ex]
 Occupational prestige & 2236.564$^{***}$ & 2221.175$^{***}$ & 2234.384$^{***}$
\\
& (182.823) & (182.515) & (183.082) \\
 Gender (woman = 1) & & -10442.232$^{**}$ & -10668.916$^{**}$ \\
& & (4922.158) & (4928.630) \\
 Age & & & -132.539$^{}$ \\
& & & (142.270) \\
 Intercept & -20009.515$^{**}$ & -13805.519$^{}$ & -7449.550$^{}$ \\
& (9120.202) & (9556.103) & (11742.412) \\
\hline \\[-1.8ex]
 Observations & 707 & 707 & 707 \\
 $R^2$ & 0.175 & 0.180 & 0.181 \\
 Adjusted $R^2$ & 0.174 & 0.178 & 0.178 \\
 Residual Std. Error & 65477.783 (df=705) & 65315.822 (df=704) & 65321.952
(df=703) \\
 F Statistic & 149.658$^{***}$ (df=1; 705) & 77.451$^{***}$ (df=2; 704) &
51.914$^{***}$ (df=3; 703) \\
\hline
\hline \\[-1.8ex]
```

```
\textit{Note:} & \multicolumn{3}{r}{$^{*}$p$<$0.1; $^{**}$p$<$0.05;
$^{***}$p$<$0.01} \\
\end{tabular}
\end{table}
```

6 Bonus Extension

Build a larger model (`lm_w`) using additional socioeconomic covariates available in the data. Compare its coefficient estimates and model-fit statistics to Models 1–3. Is it a better model? Justify your choice.

[14]:
```
lm_w = smf.ols('income_cont ~ prestg10 + sex_woman + age + working', data=df).
  ↪fit()
print(lm_w.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:            income_cont   R-squared:                       0.208
Model:                            OLS   Adj. R-squared:                  0.204
Method:                 Least Squares   F-statistic:                     46.22
Date:                Wed, 28 Oct 2026   Prob (F-statistic):           1.73e-34
Time:                        02:37:52   Log-Likelihood:                -8827.9
No. Observations:                 707   AIC:                         1.767e+04
Df Residuals:                     702   BIC:                         1.769e+04
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025
0.975]
------------------------------------------------------------------------------
Intercept      -4.096e+04   1.34e+04     -3.051      0.002   -6.73e+04
-1.46e+04
sex_woman[T.Woman] -1.058e+04   4849.897     -2.181      0.030   -2.01e+04
-1055.371
working[T.Working]  2.825e+04   5763.704      4.901      0.000    1.69e+04
3.96e+04
prestg10            2134.3988    181.307     11.772      0.000    1778.429
2490.368
age                  295.7130    165.027      1.792      0.074     -28.292
619.718
==============================================================================
Omnibus:                      111.980   Durbin-Watson:                   1.689
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              165.110
Skew:                           1.104   Prob(JB):                     1.40e-36
Kurtosis:                       3.854   Cond. No.                         415.
==============================================================================
```

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

[ ]:
```
comparison3 = pd.DataFrame({
        'Model':          ['Model 1', 'Model 2', 'Model 3', 'Model 4'],
        'R-Squared':      [lm1.rsquared, lm2.rsquared,     lm3.rsquared, lm_w.
 ↪rsquared],
        'Adj. R-Squared': [lm1.rsquared_adj, lm2.rsquared_adj, lm3.
 ↪rsquared_adj, lm_w.rsquared_adj],
        'AIC':            [lm1.aic, lm2.aic,          lm3.aic, lm_w.aic]
})
comparison3

# model w explains 20% of the variation in income. it has a higher R-squared␣
 ↪than the other models, so model 4 is slightly better in explaining variation␣
 ↪in income
# model w has a higher adjusted R-squared than other models. adding work helps␣
 ↪a little in explaining variation in income
# model w has a lower AIC value, so it fits the data better than the other␣
 ↪models
```

[ ]:

	Model	R-Squared	Adj. R-Squared	AIC
0	Model 1	0.175109	0.173939	17688.881424
1	Model 2	0.180349	0.178020	17686.375978
2	Model 3	0.181360	0.177866	17687.503692
3	Model 4	0.208445	0.203935	17665.716121

[ ]:
```
print(anova_lm(lm1, lm2, lm3, lm_w))

# since p-value = 0.000001 < 0.05, adding work is statistically significant.
# model 5 is significantly better
```

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	705.0	3.022575e+12	0.0	NaN	NaN	NaN
1	704.0	3.003374e+12	1.0	1.920049e+10	4.647164	0.031443
2	703.0	2.999671e+12	1.0	3.703232e+09	0.896307	0.344099
3	702.0	2.900424e+12	1.0	9.924730e+10	24.021182	0.000001

[17]:
```
!jupyter nbconvert --to pdf week5_notebook_student.ipynb
```

```
[NbConvertApp] Converting notebook week5_notebook_student.ipynb to pdf
[NbConvertApp] Writing 49978 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
```

```
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 70224 bytes to week5_notebook_student.pdf
```